

What is Spring Data?

Spring Data is a part of the **Spring Framework** that makes it easier to work with data access layers, especially **relational databases** (like MySQL, PostgreSQL), **NoSQL databases** (like MongoDB), and other storage systems.

Specifically, **Spring Data JPA** is the module that simplifies **JPA (Java Persistence API)** development — so you don't have to write boilerplate code like EntityManager, transactions, or long SQL queries.

Benefits of Spring Data

- No need to write SQL/JPQL for basic CRUD.
- Auto-generates queries based on method names.
- Easily supports paging, sorting, and filtering.
- Allows use of JPQL, Native SQL, Specifications, etc.

Ways to Fetch Data Using Spring Data

- There are 5 **common ways** to fetch data using **Spring Data JPA**:

1. Derived Query Methods (Method Naming Convention)

- Write method names following specific rules and Spring auto-generates the SQL.

```
List<Product> findByName(String name);
```

```
List<Product> findByPriceLessThan(Double price);
```

```
List<Product> findByCategoryAndStock(String  
category, int stock);
```

Spring will automatically generate the SQL behind the scenes.

2. JPQL (Java Persistence Query Language)

Use the `@Query` annotation to write queries using entity names and field names.

```
@Query("SELECT p FROM Product p WHERE p.stock  
> :stock")
```

```
List<Product>
```

```
findProductsWithStockMoreThan(@Param("stock")  
int stock);
```

Uses entity class (Product) and fields (stock), not table/column names.

3. Native SQL Queries

Use the `@Query` annotation with `nativeQuery = true` to run actual SQL queries.

```
@Query(value = "SELECT * FROM products WHERE  
category = :category", nativeQuery = true)
```

```
List<Product>
```

```
findProductsByCategory(@Param("category")
```

```
String category);
```